Báo cáo LAB-03 – Debugging Kernel & Phân tích Hiệu năng

Nguyễn Ngọc Quang – SE202007

22/10/2026

I. Thông tin & môi trường

Lab 3 hoàn thành đủ 4 phần A–D trên QEMU vexpress-a9 (ARMv7, Cortex-A9) và máy host x86; mã nguồn, log, ảnh năm ở repo GitHub Embedded-linux-SE202007-NguyenNgocQuang, thư mục lab03/.

Mục	Nội dung
Họ tên / MSSV	Nguyễn Ngọc Quang / SE202007
Môn học	Embedded Linux Systems (LOS201)

II. Phần A – GDB remote debug (buggy_app)

buggy_app chạy trong QEMU qua gdbserver 0.0.0.0:1234; trên host nối bằng set architecture arm + target remote localhost:1234, đặt breakpoint ở main, process_data, create_message, calculate. Tìm được 3 lỗi:

#	Loại lỗi	Vị trí	Giá trị quan sát được	Sửa
1	Out-of-bounds read	process_data, buggy_app.c:8	i == len == 5 vẫn vào vòng lặp, data[5] tại 0x7efffd24 = 348751360 (rác)	i <= len → i < len
2	Use-after-free	create_message, buggy_app.c:18–19	sau free(buf) vẫn return buf; x/s buf từ "Hello Embedded Linux" thành "u"	Không free trong hàm; nơi gọi free sau khi dùng xong
3	Integer overflow	calculate, buggy_app.c:24	100000 * 100000 ra 1410065408 (0x540be400), đúng là 10000000000	Dùng long long (int64_t) cho result

Kết nối tới gdbserver và backtrace tại process_data (frame #1 là main, buggy_app.c:34):

```
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
Breakpoint 1, main (argc=1, argv=0x7efffe94) at buggy_app.c:29
29      int main(int argc, char *argv[]) {
(gdb) bt
#0  main (argc=1, argv=0x7efffe94) at buggy_app.c:29
(gdb) info locals
arr = {306564, 2130705952, 1996484960, 92331600, 0}
msg = 0x73734 <_dl_debug_mask> ""
(gdb) continue
Continuing.

Breakpoint 2, process_data (data=0x7efffd10, len=5) at buggy_app.c:7
7           int sum = 0;
(gdb) bt
#0  process_data (data=0x7efffd10, len=5) at buggy_app.c:7
#1  0x00010450 in main (argc=1, argv=0x7efffe94) at buggy_app.c:34
(gdb) info args
data = 0x7efffd10
len = 5
(gdb)
```

Kết nối target remote, breakpoint và backtrace

Bug 1 – đọc vượt mảng. Breakpoint có điều kiện break buggy_app.c:9 if i == 5 dừng đúng lúc i bằng len. Đặt watchpoint trên i cho thấy i tăng 5 → 6 ở dòng 8, tức vòng lặp chạy thêm một lần vì dùng <=.

```
Breakpoint 5 at 0x10368: file buggy_app.c, line 9.
(gdb) continue
Continuing.

Breakpoint 5, process_data (data=0x7efffd10, len=5) at buggy_app.c:9
9               sum += data[i];               /* Out-of-bounds khi i==len */
(gdb) print i
$1 = 5
(gdb) print len
$2 = 5
(gdb) print data[5]
$3 = 348751360
(gdb) print &data[5]
$4 = (int *) 0x7efffd24
(gdb) x/6dw data
0x7efffd10:     10      20      30      40
0x7efffd20:     50      348751360
(gdb) info registers pc sp lr
pc             0x10368            0x10368 <process_data+20>
sp             0x7efffce8         0x7efffce8
lr             0x10451            66641
(gdb) delete 6
(gdb) set can-use-hw-watchpoints 0
(gdb) watch i
Watchpoint 7: i
(gdb) continue
Continuing.

Watchpoint 7: i

Old value = 5
New value = 6
0x0001037e in process_data (data=0x7efffd10, len=5) at buggy_app.c:8
```

Bug 1: breakpoint có điều kiện, data[5] ngoài mảng, info registers và watchpoint trên i

QEMU không hỗ trợ hardware watchpoint (Could not insert hardware watchpoint), nên phải bật set can-use-hw-watchpoints 0 để dùng software watchpoint.

Bug 2 – use-after-free. buf vẫn in đúng ngay sau strcpy, nhưng sau free(buf) heap bị tái sử dụng nên nội dung hỏng, trong khi hàm vẫn trả con trỏ đó về cho main.

```
Delete all breakpoints, watchpoints, tracepoints, and catchpoints? (y or n) y
(gdb) break create_message
Breakpoint 8 at 0x103a8: file buggy_app.c, line 16.
(gdb) continue
Continuing.

Breakpoint 8, create_message (text=0x491ac "Hello Embedded Linux") at buggy_app.c:16
16          char *buf = malloc(strlen(text) + 1);
(gdb) next
17          strcpy(buf, text);
(gdb) next
18          free(buf);
(gdb) print buf
$5 = 0x75c38 "Hello Embedded Linux"
(gdb) x/s buf
0x75c38:        "Hello Embedded Linux"
(gdb) next
19          return buf;   /* Bug: tra ve pointer da bi free */
(gdb)
print buf
❌ Undefined command: "".  Try "help".
(gdb) x/s buf
0x75c38:        "u"
(gdb) x/6xw buf
0x75c38:        0x00000075      0xc482df99      0x64646562      0x4c206465
0x75c48:        0x78756e69      0x00000000
(gdb)
```

Bug 2: nội dung buf trước và sau free

Bug 3 – tràn số nguyên. info args cho a = b = 100000; phép nhân 32-bit bị cắt, còn print (long long)a * b cho giá trị đúng.

```
(gdb) break calculate
Breakpoint 9 at 0x103de: file buggy_app.c, line 24.
(gdb) continue
Continuing.

Breakpoint 9, calculate (a=100000, b=100000) at buggy_app.c:24
24          int result = a * b;   /* Bug: a*b co the overflow int */
(gdb) info args
a = 100000
b = 100000
(gdb) next
25          printf("calculate(%d, %d) = %d\n", a, b, result);
(gdb) print result
$6 = 1410065408
(gdb) print (long long)a * b
$7 = 10000000000
(gdb) print/x result
$8 = 0x540be400
(gdb)
```

*Bug 3: a*b tràn int, so với phép tính long long*

III. Phần B – Phát hiện rò rỉ bộ nhớ (leaky_app)

Valgrind memcheck (--leak-check=full --track-origins=yes, build x86 với -g -O0) báo leaky_app rò 16.900 byte trong 250 block, toàn bộ là *definitely lost*: 352 lần malloc nhưng chỉ 102 lần free.

```
==6384==     in use at exit: 16,900 bytes in 250 blocks
==6384==   total heap usage: 352 allocs, 102 frees, 21,124 bytes allocated
==6384== 2,000 bytes in 100 blocks are definitely lost in loss record 1 of 3
==6384== 2,100 bytes in 100 blocks are definitely lost in loss record 2 of 3
==6384== 12,800 bytes in 50 blocks are definitely lost in loss record 3 of 3
==6384==    definitely lost: 16,900 bytes in 250 blocks
==6384==    indirectly lost: 0 bytes in 0 blocks
==6384==      possibly lost: 0 bytes in 0 blocks
==6384==    still reachable: 0 bytes in 0 blocks
==6384== ERROR SUMMARY: 3 errors from 3 contexts (suppressed: 0 from 0)
```

LEAK SUMMARY của Valgrind trên leaky_app

Loss record	Byte / block	Nguồn cấp phát	Nguyên nhân
1	2.000 / 100	create_record (leaky_app.c:19)	delete_record không free(r->scores)
2	2.100 / 100	create_record (leaky_app.c:17)	delete_record không free(r->name)
3	12.800 / 50	process_messages (leaky_app.c:33)	thiếu free(buf) trong vòng lặp

```
==6384== 2,000 bytes in 100 blocks are definitely lost in loss record 1 of 3
==6384==    at 0x4850858: malloc (vg_replace_malloc.c:447)
==6384==    by 0x4001280: create_record (leaky_app.c:19)
==6384==    by 0x4001414: main (leaky_app.c:46)
==6384==
==6384== 2,100 bytes in 100 blocks are definitely lost in loss record 2 of 3
==6384==    at 0x4850858: malloc (vg_replace_malloc.c:447)
==6384==    by 0x400124D: create_record (leaky_app.c:17)
==6384==    by 0x4001414: main (leaky_app.c:46)
==6384==
==6384== 12,800 bytes in 50 blocks are definitely lost in loss record 3 of 3
==6384==    at 0x4850858: malloc (vg_replace_malloc.c:447)
==6384==    by 0x4001314: process_messages (leaky_app.c:33)
==6384==    by 0x4001487: main (leaky_app.c:54)
==6384==
==6384== LEAK SUMMARY:
```

Stack trace của 3 loss record

Sau khi sửa (thêm free(r->name), free(r->scores), free(buf)): 352 allocs / 352 frees, "All heap blocks were freed", ERROR SUMMARY 0 lỗi.

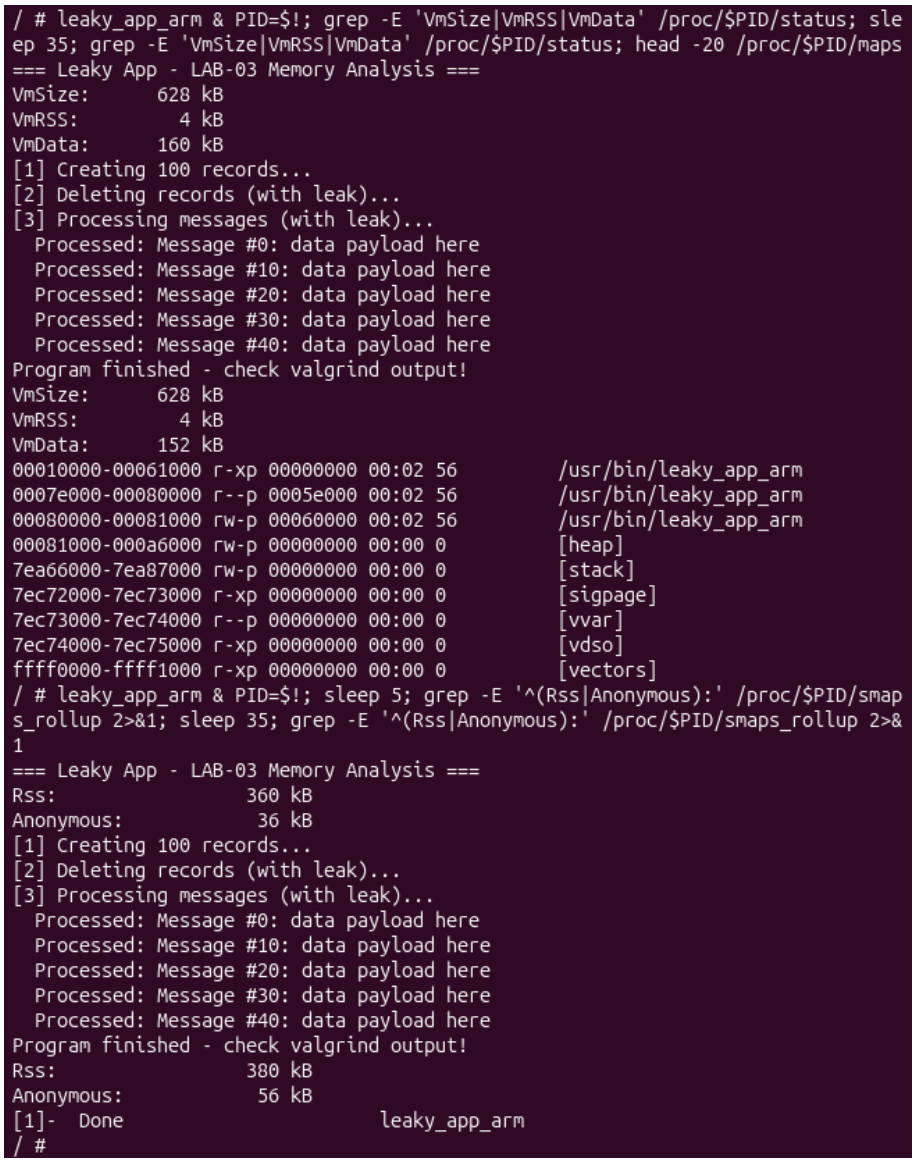
```
==6544==   total heap usage: 352 allocs, 352 frees, 21,124 bytes allocated
==6544== All heap blocks were freed -- no leaks are possible
==6544== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Valgrind sau khi sửa: không còn leak

Theo dõi trên ARM (QEMU). Chạy leaky_app_arm trong QEMU (bản có sleep 30 s ở đầu và cuối main) và đo hai lần:

Chỉ số	Trước	Sau
VmSize (kB)	628	628
VmRSS (kB)	4	4
VmData (kB)	160	152
smaps_rollup: Rss (kB)	360	380
smaps_rollup: Anonymous (kB)	36	56

/proc/PID/status hầu như không đổi nên không đủ để thấy leak ~17 kB; smaps_rollup mới cho thấy Anonymous tăng 20 kB (khớp với 16.900 byte + overhead của malloc làm tròn theo trang). Vùng [heap] trong /proc/PID/maps là 00081000-000a6000 (148 kB).



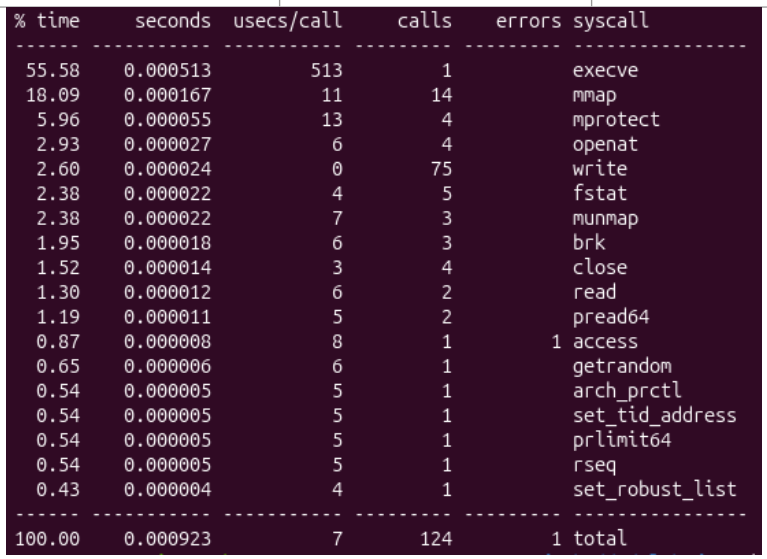
/proc/PID/status, maps và smaps_rollup của leaky_app_arm

IV. Phần C – strace & perf (perf_target)

perf_target gồm 3 pha: tính toán (heavy_compute dùng log()), tìm kiếm tuyến tính 1000 lần, ghi 10.000 dòng ra file. Kết quả: chương trình bị chậm bởi CPU (gọi log() trong libm), không phải bởi I/O hay syscall.

strace -c (host x86, 124 syscall, tổng 0,923 ms):

Hạng	Syscall	% thời gian	Số lần gọi
1	execve	55,58	1
2	mmap	18,09	14
3	mprotect	5,96	4
4	openat	2,93	4
5	write	2,60	75



% time	seconds	usecs/call	calls	errors	syscall
55.58	0.000513	513	1		execve
18.09	0.000167	11	14		mmap
5.96	0.000055	13	4		mprotect
2.93	0.000027	6	4		openat
2.60	0.000024	0	75		write
2.38	0.000022	4	5		fstat
2.38	0.000022	7	3		munmap
1.95	0.000018	6	3		brk
1.52	0.000014	3	4		close
1.30	0.000012	6	2		read
1.19	0.000011	5	2		pread64
0.87	0.000008	8	1	1	access
0.65	0.000006	6	1		getrandom
0.54	0.000005	5	1		arch_prctl
0.54	0.000005	5	1		set_tid_address
0.54	0.000005	5	1		prlimit64
0.54	0.000005	5	1		rseq
0.43	0.000004	4	1		set_robust_list
100.00	0.000923	7	124	1	total

strace -c summary

perf record -g / perf report (117 mẫu, perf 7.0.14), top 5 theo Self:

Hạng	Hàm	Shared object	Self (%)
1	__ieee754_log_fma	libm.so.6	56,55
2	heavy_compute	perf_target	23,28
3	__log_finite@GLIBC_2.15@plt	libm.so.6	6,41
4	log@@GLIBC_2.29	libm.so.6	6,30
5	log@plt	perf_target	3,70

```
    56.55%  perf_target  libm.so.6             [.] __ieee754_log_fma
    23.28%  perf_target  perf_target           [.] heavy_compute
     6.41%  perf_target  libm.so.6             [.] __log_finite@GLIBC_2.15@plt
     6.30%  perf_target  libm.so.6             [.] log@@GLIBC_2.29
     3.70%  perf_target  perf_target           [.] log@plt
     0.98%  perf_target  [kernel.kallsyms]     [k] native_irq_return_iret
     0.57%  perf_target  [kernel.kallsyms]     [k] lru_add
     0.53%  perf_target  perf_target           [.] main
     0.48%  perf_target  [kernel.kallsyms]     [k] __rmqueue_pcplist
     0.41%  perf_target  [kernel.kallsyms]     [k] lruvec_stat_mod_folio
```

perf report top 5

perf stat: 103,25 triệu cycles, 367,36 triệu instructions (IPC 3,6), branch-miss 0,0 %, 376 page-fault, 2 context-switch; Top-down: retiring 70,9 %, backend-bound 22,2 %, bad-speculation 5,1 %, frontend-bound 1,8 %.

```
Performance counter stats for './perf_target':

               2      context-switches                 #     77.3 cs/sec  cs_per_second
               0      cpu-migrations                   #      0.0 migrations/sec  migrations_per_second
             376      page-faults                      #  14537.6 faults/sec  page_faults_per_second
           25.86 msec task-clock                       #      0.2 CPUs  CPUs_utilized
           6,523      branch-misses                    #      0.0 %  branch_miss_rate         (88.40%)
      50,462,627      branches                         #   1951.1 M/sec  branch_frequency     (88.41%)
     103,254,349      cpu-cycles                       #      4.0 GHz  cycles_frequency       (88.40%)
     367,360,683      instructions                     #      3.6 instructions  insn_per_cycle  (88.40%)
                      TopdownL1                        #     22.2 %  tma_backend_bound
                                                       #      5.1 %  tma_bad_speculation      (88.40%)
                                                       #      1.8 %  tma_frontend_bound       (81.18%)
                                                       #     70.9 %  tma_retiring             (91.23%)

     0.120612107 seconds time elapsed

     0.024709000 seconds user
     0.001976000 seconds sys
```

perf stat

Nhận xét bottleneck. Khoảng 73 % mẫu nằm trong log() và thêm 23 % trong heavy_compute; thời gian syscall chỉ ~0,9 ms. IPC cao và retiring 70,9 % cho thấy CPU đang bận tính toán chứ không chờ, đây là **CPU-bound**. Muốn nhanh hơn cần giảm số lần gọi log() (tính trước, dùng bảng tra, hoặc bật -O2) thay vì tối ưu I/O.

So sánh ARM với host. perf_target_arm (static ARMv7) chạy trong QEMU mất real 0,93 s (user 0,91 s, sys 0,01 s), trong khi trên host perf stat ghi elapsed 0,12 s và task-clock 25,9 ms. ARM chậm hơn đáng kể vì QEMU đang giả lập CPU bằng phần mềm; con số chỉ mang tính tham khảo vì lần chạy perf trên host báo segmentation fault ở cuối chương trình (dữ liệu đã được ghi trước đó). Không chạy được strace/perf trên ARM vì rootfs LAB-02 không có các công cụ này.

```
/ # busybox time perf_target_arm
=== Perf Target App - LAB-03 ===
[1] CPU-intensive computation...
    Result: 11433083166.074631
[2] Linear search (1000 times)...
[3] I/O intensive (write 10000 lines)...
Done.
real    0m 0.93s
user    0m 0.91s
sys     0m 0.01s
/ #
```

busybox time perf_target_arm trong QEMU

V. Phần D – Đo độ trễ real-time bằng cyclictest

Kernel host là Linux 7.0.0-38-generic (x86_64, 8 lõi) với PREEMPT_DYNAMIC=y, PREEMPT_RT chưa bật; đây là mốc "standard kernel".

```
quang@quang-VivoBook-ASUSLaptop-X513EAN-A515EA:~/embedded-linux-SE202007$ uname -a
Linux quang-VivoBook-ASUSLaptop-X513EAN-A515EA 7.0.0-38-generic #38-Ubuntu SMP PREEMPT_DYNAMIC Fri Sep  4 09:10:14 UTC 2026 x86_64 GNU/Linux
quang@quang-VivoBook-ASUSLaptop-X513EAN-A515EA:~/embedded-linux-SE202007$ grep -E 'CONFIG_PREEMPT' /boot/config-$(uname -r) | head -10
CONFIG_PREEMPT_BUILD=y
# CONFIG_PREEMPT is not set
CONFIG_PREEMPT_LAZY=y
# CONFIG_PREEMPT_RT is not set
CONFIG_PREEMPT_COUNT=y
CONFIG_PREEMPTION=y
CONFIG_PREEMPT_DYNAMIC=y
CONFIG_PREEMPT_RCU=y
CONFIG_PREEMPT_NOTIFIERS=y
# CONFIG_PREEMPT_TRACER is not set
quang@quang-VivoBook-ASUSLaptop-X513EAN-A515EA:~/embedded-linux-SE202007$ nproc
```

uname và cấu hình CONFIG_PREEMPT của kernel

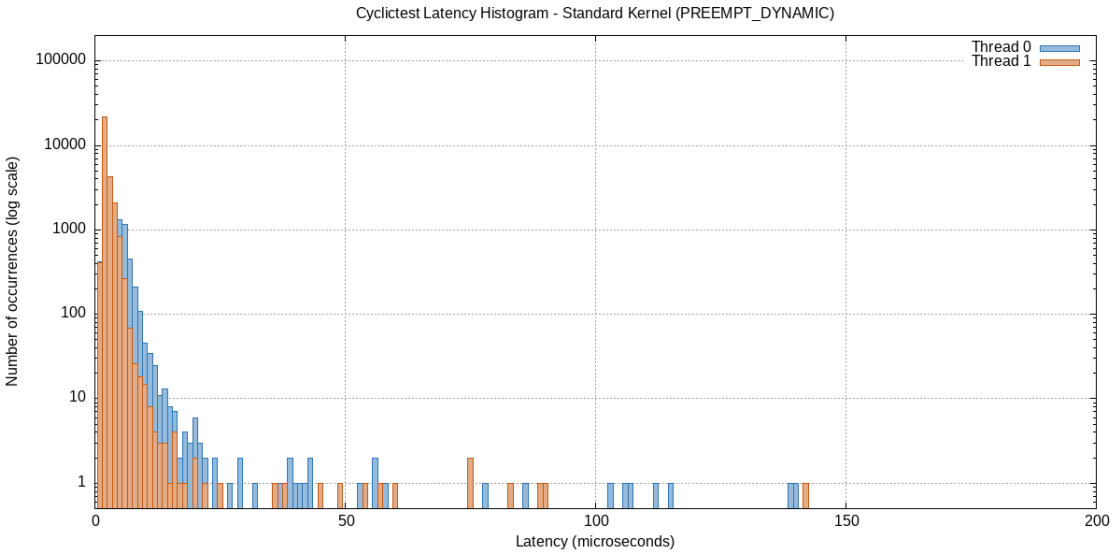
Lệnh: cyclictest --mlockall --threads=2 --priority=80 --interval=1000 --loops=30000 --histogram=200 (SCHED_FIFO, chu kỳ 1 ms).

Thread	Chu kỳ đo	Min (µs)	Avg (µs)	Max (µs)
0	30.000	1	2	140
1	30.000	1	2	142

```
T: 0 ( 6576) P:80 I:1000 C:  30000 Min:      1 Act:    4 Avg:    2 Max:      140
T: 1 ( 6577) P:80 I:1000 C:  30000 Min:      1 Act:    3 Avg:    2 Max:      142
# Min Latencies: 00001 00001
# Avg Latencies: 00002 00002
# Max Latencies: 00140 00142
# Histogram Overflows: 00000 00000
# Histogram Overflow at cycle number:
# Thread 0:
# Thread 1:
```

Kết quả cyclictest và phần cuối hist_standard.txt

99,95 % mẫu của thread 0 nằm dưới 50 µs; phần còn lại là vài mẫu rời rạc kéo đuôi đến ~140 µs (trục Y theo log).



Histogram độ trễ cyclictest

Max lớn hơn Avg ~70 lần vì trung bình bị hàng chục nghìn mẫu tốt che đi, còn Max chỉ cần một lần bị ngắt (interrupt, vùng kernel không preempt được, tiến trình khác, quản lý nguồn/SMI của laptop). Với hệ real-time chỉ số xấu nhất (worst-case) mới quyết định deadline có bị lỡ hay không.

PREEMPT_RT biến hầu hết spinlock thành mutex có thể preempt, chạy IRQ handler trong thread, nên tiến trình ưu tiên cao gần như luôn chạy ngay khi sẵn sàng; nhờ đó đuôi phân bố bị chặn trên. Em chưa chạy so sánh thực nghiệm trên kernel RT/lowlatency (máy host không ổn định khi tải nặng), nên phần này chỉ dừng ở lý thuyết.

Đề xuất cho robot: vòng điều khiển 1 kHz nên chạy trên kernel PREEMPT_RT, thread SCHED_FIFO kèm mlockall, gán riêng một lõi CPU (isolcpus), và nghiệm thu bằng cyclictest dưới tải nặng, so Max với chu kỳ điều khiển.

VI. Trả lời câu hỏi phân tích

Câu 1 – Chuỗi sự kiện khi gõ next. GDB (client) dùng bảng dòng/địa chỉ trong debug info để biết dòng hiện tại kéo dài đến địa chỉ nào, rồi gửi lệnh tiếp tục/bước từng lệnh (ARM không có single-step phần cứng nên GDB tự tính địa chỉ lệnh kế và đặt breakpoint tạm) qua TCP :1234, đi xuyên hostfwd của QEMU tới gdbserver trong guest. gdbserver gọi ptrace(PTRACE_CONT/SINGLESTEP) lên tiến trình buggy_app, waitpid đợi SIGTRAP, rồi trả gói dừng kèm thanh ghi; GDB đọc thêm thanh ghi/bộ nhớ để cập nhật frame và in dòng mới. Giao thức là **GDB Remote Serial Protocol (RSP)**, gói dạng $lệnh#checksum chạy trên TCP. Phải dùng gdb-multiarch vì gdb thường chỉ được build cho x86_64, không hiểu tập lệnh, thanh ghi và ABI của ARM (trong lab phải gõ set architecture arm).

Câu 2 – definitely vs indirectly lost. *Definitely lost*: không còn con trỏ nào trỏ tới block. *Indirectly lost*: block chỉ còn được trỏ bởi một block khác đã bị *definitely lost* (ví dụ cả struct cha bị rò). Trong delete_record(), free(r) đã giải phóng struct cha, nên r->name và r->scores mất con trỏ cuối cùng mà không còn block cha nào bị rò; do đó chúng là **definitely lost**. Đúng

với kết quả thực tế: definitely lost 16.900 byte, indirectly lost 0 byte. Ngoài leak, Valgrind còn báo: đọc/ghi ngoài vùng heap (invalid read/write, như bug 1), dùng bộ nhớ đã free (như bug 2), dùng giá trị chưa khởi tạo, free hai lần/free sai con trỏ, và memcpy có vùng chồng lấn.

Câu 3 – Tối ưu heavy_compute(). perf cho thấy ~73 % mẫu nằm trong log() và IPC đã cao (3,6), nên nút thắt là số phép tính, không phải cache. Các số dưới đây là **ước tính**, chưa đo:

1. *Không tính lại dữ liệu không đổi.* Mảng darr không đổi qua 50 vòng, nên chỉ cần tính sqrt(x)*log(x+1) một lần (lưu bảng hoặc cache kết quả). Phần tính toán giảm gần 50 lần.
2. *NEON trên ARMv7.* Đổi sang float và dùng NEON (-mfpu=neon-vfpv3, intrinsic hoặc tự vector hóa với -O3 -ffast-math) để xử lý 4 phần tử mỗi lệnh; kỳ vọng nhanh hơn khoảng 2–4 lần, đánh đổi là mất độ chính xác (float 24 bit) và NEON không hỗ trợ double.
3. *Giảm chi phí gọi hàm.* log@plt và __log_finite@plt chiếm ~10 % mẫu; liên kết tĩnh/-fno-plt và bật -O2 có thể giảm vài chục phần trăm. Loop unrolling và cải thiện cache locality ít có tác dụng vì mảng được duyệt tuần tự và đang bị chặn bởi phép tính.

Câu 4 – Vì sao Max (140–142 µs) cao hơn Avg (2 µs) hàng chục lần.

Nguyên nhân trên standard kernel	PREEMPT_RT xử lý
Ngắt (interrupt/softirq): handler chạy với preemption tắt, thread RT phải chờ	IRQ handler chạy trong kernel thread có độ ưu tiên, softirq cũng đưa vào thread nên bị thread RT ưu tiên hơn preempt
Scheduler/vùng kernel không preempt được (spinlock, preempt_disable): thread SCHED_FIFO không thể chen vào	Spinlock thành rt_mutex có thể ngủ và có priority inheritance, hầu hết kernel preempt được
Quản lý bộ nhớ: page fault, thu hồi trang, kernel giữ lock lâu	các lock trong mm cũng preempt được; vẫn cần mlockall (đã dùng trong lệnh cyclictest) để tránh page fault
Phần cứng/BIOS (SMI, C-state, đổi xung CPU)	Không xử lý được, phải cấu hình BIOS/CPU

Histogram cho thấy đúng đặc trưng này: gần như toàn bộ mẫu dưới 50 µs, phần đuôi dài chỉ vài mẫu (log scale).

Câu 5 – Chọn công cụ theo vấn đề.

Vấn đề	Công cụ	Vì sao, dẫn chứng từ lab
(a) Crash ngẫu nhiên sau vài giờ	GDB (bổ sung Valgrind)	catch signal SIGSEGV/core dump cho backtrace và giá trị biến tại điểm lỗi; trong lab GDB lộ ra cả 3 lỗi (out-of-bounds, use-after-free, overflow)

Vấn đề	Công cụ	Vì sao, dẫn chứng từ lab
(b) Chạy chậm dần theo thời gian	Valgrind (theo dõi bằng smaps_rollup)	dấu hiệu điển hình của leak; lab: 16.900 byte definitely lost, Anonymous tăng 20 kB
(c) Đôi khi trễ deadline	cyclictest	đo trực tiếp worst-case latency và đuôi histogram; lab: Max 140–142 µs trong khi Avg chỉ 2 µs
(d) Một hàm làm cả hệ thống chậm	perf (strace để loại trừ I/O)	perf chỉ đúng hàm nóng; lab: log() + heavy_compute chiếm ~96 % mẫu, trong khi syscall chỉ tốn ~0,9 ms

VII. Nhận xét & khó khăn gặp phải

Bốn công cụ trả lời bốn loại câu hỏi khác nhau: GDB cho lỗi logic, Valgrind cho lỗi bộ nhớ, perf/strace cho hiệu năng, cyclictest cho độ trễ worst-case. Chưa làm: flame graph (bonus), strace trên ARM (rootfs không có), và đo thực tế trên kernel lowlatency/RT.

Khó khăn	Cách giải quyết
Build gdbserver ARM: configure trong thư mục gdbserver/ của GDB 17.1 báo thiếu Makefile.gnulib.inc	Build từ thư mục gốc với --enable-gdbserver --disable-gdb, liên kết tĩnh để chạy được trong rootfs không có libc
QEMU không có hardware watchpoint (Could not insert hardware watchpoint)	set can-use-hw-watchpoints 0 để dùng software watchpoint
/proc/PID/status không cho thấy RSS tăng, khó chứng minh leak	Đo thêm smaps_rollup (Rss, Anonymous) và thêm sleep để kịp đo lúc đầu/cuối
Máy host tự tắt nhiều lần khi biên dịch/chạy QEMU, phải làm lại bước đang dở	Giảm tải, ghi lại từng lệnh/log vào logs/ và push lên GitHub sau mỗi phần
perf record/perf stat báo segmentation fault sau khi thu dữ liệu	Dữ liệu đã được ghi nên perf report vẫn đọc được; ghi chú trong báo cáo